QueryGenie: A Self-Correcting Natural Language to SQL Interface

A report submitted in partial fulfillment of the requirements
for the award of the degree of

Bachelor of Technology
in
Computer Science & Information Technology

by

N V V S S Jayakanth Kamisetti	**2320090050**
Boddupally Narendar	**2320090009**
M Praneeth Reddy	**2320090018**

Under the Supervision of
M Parameswar



KONERU LAKSHMAIAH EDUCATION FOUNDATION, BOWRAMPET

Hyderabad-500043, Telangana, India

2026

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

CERTIFICATE

This is to certify that **N V V S S Jayakanth Kamisetti, Boddupally Narendar and M Praneeth Reddy** (ID.No. **2320090050, 2320090009 and 2320090018**), B.Tech (Computer Science & Information Technology), have worked on the project entitled **"QueryGenie: A Self-Correcting Natural Language to SQL Interface"** under my supervision and guidance. The content of this report is original and has not been submitted elsewhere for the award of any academic or professional degree.

.....

M Parameswar
Designation,
Department of Computer Science & Information Technology,
Koneru Lakshmaiah Education Foundation, Telangana, India
Bowrampet, Hyderabad-500043

Date: .....

Acknowledgments

We thank our supervisor, M Parameswar, for the guidance given throughout this project, and the Department of Computer Science & Information Technology, Koneru Lakshmaiah Education Foundation, for the structure and review process that shaped it.

We are grateful to the authors of CodeS for releasing their code, their fine-tuned model weights and their evaluation setup openly; without them a faithful reproduction would not have been possible. We also thank the creators of the Spider benchmark and of its test-suite evaluator, and the open-source communities behind the tools this work builds on.

Abstract

Text-to-SQL models let people who do not know SQL query databases, but they answer every question with the same apparent confidence, so a wrong answer looks exactly like a right one. This project reproduces CodeS, an open-source text-to-SQL model (SIGMOD 2024), and extends it with confidence-aware abstention.

Reproduction. Using the released fine-tuned CodeS-1B checkpoint and the authors' database prompt and inference procedure, we obtain 77.8% Execution Accuracy on the 1,034-question Spider development set, scored with the official evaluator, against 77.9% reported in the paper — a difference of 0.1 points. An audit of our harness against the authors' code found the simplified metric and value-retriever bug behind earlier results of 71.7–73.8%. An ablation shows every prompt component contributes 0.9 to 1.9 points.

Enhancement. The system executes the four candidate queries produced by beam search and measures how many return the same result. When agreement is low it declines and asks the user to choose between readings. At a pre-registered operating point it answers 90.4% of questions and is correct on 83.3% of them, against 77.8% when answering everything; wrong answers fall from 230 to 156 (-32%). The gain is significant (paired t-test $p = 7.7 \times 10^{-10}$; Wilcoxon $p = 9.8 \times 10^{-4}$; bootstrap 95% CI +4.5 to +6.7 points).

Application. QueryGenie, a desktop application for macOS and Windows, runs the evaluated pipeline locally and shows, for every answer, how many candidate queries agree and the SQL behind it — and, when unsure, the readings to choose from.

Keywords: text-to-SQL, reproducibility, large language models, selective prediction, abstention.

Contents

Acknowledgments **ii**

List of Tables **vi**

List of Figures **viii**

1 Introduction **1**

1.1 Problem 1

1.2 Motivation 2

1.3 Objectives 2

1.4 Contributions 3

1.5 Scope 3

1.6 Organisation of the report 4

2 Literature survey **5**

2.1 Surveys 5

2.2 Benchmark datasets 6

2.3 Neural and pre-trained approaches 6

2.4 Methods based on large language models 7

2.5 Presenting results and knowing when to abstain 7

2.6 Findings and the gap addressed 8

3 Data Collection and Preliminary Analysis 9

3.1 The Spider benchmark 9

3.2 Data collection 9

3.3 Size and composition 10

3.4 Preliminary analysis 11

3.4.1 What the questions ask for 11

3.4.2 Difficulty 11

3.5 Preparing the input for the model 12

3.6 Data used by the application 13

4 Proposed Approach 14

4.1 Architecture 14

4.2 Reproduction methodology 14

4.2.1 The CodeS pipeline 14

4.2.2 Environment 15

4.2.3 The reproduced pipeline 16

4.2.4 Audit of the evaluation harness 17

4.2.5 Run logging and determinism 18

4.3 Reproducibility results 18

4.3.1 Run history 18

4.3.2 Headline comparison 19

4.3.3 Ablation study 19

4.4 Enhancement: confidence-aware abstention 20

4.4.1 Choice and justification 20

4.4.2 Method 21

4.4.3 Hypothesis and protocol 21
4.4.4 Results 23
4.4.5 Limits of the enhancement 23
4.5 The application 25
4.6 Differences from the authors' setup 26
4.7 Source code and how to run it 28

5 Conclusion and Future Scope of work **31**
5.1 Conclusion 31
5.2 Future scope of work 32

Index **34**

Bibliography **34**

Biography **40**

List of Tables

2.1 Results reported in the CodeS paper (its Table 5), Spider development set. 7

3.1 Spider splits used in this project. 10

3.2 SQL features in the development-set gold queries (a query can have several). 11

3.3 Development set by official difficulty level, and the reproduced CodeS-1B Execution Accuracy (run T2-006). 12

4.1 The authors' environment and ours. 16

4.2 Runs on the Spider development set (1,034 questions). "Simple" is the early sorted-rows metric; only "official" matches the paper. 19

4.3 Ablation study, official evaluator. 19

4.4 Original against enhanced on the same 1,034 questions. 23

4.5 Significance tests over 10 calibration/test splits (mean coverage 90.5%) and a 5,000-sample bootstrap over questions. 23

4.6 Accuracy by the number of candidates agreeing with the answer. 24

4.7 Remaining deviations from the authors' setup. 27

List of Figures

4.1 The pipeline as evaluated: reproduced CodeS components (blue), the enhancement (amber) and the application (green). 15

4.2 The CodeS pipeline, annotated from the paper (Week-3 deliverable). . 16

4.3 Drop in Execution Accuracy when each component is removed. 20

4.4 Accuracy on answered questions against coverage. The dashed line is the original system (and random abstention); dots mark 90% coverage. 24

4.5 An answer with its confidence: two of the four candidate queries agree, so the answer is shown as low confidence, with the other reading offered. 27

4.6 Abstention under the strict setting: the candidates split two against two on the flagship question, so the application asks which reading was meant instead of guessing. 28

4.7 The schema map: tables, columns, primary keys and foreign keys of a Spider database. 29

4.8 The model picker below the question box: the evaluated model, the earlier prototype's models and local Ollama models. 30

Chapter 1

Introduction

1.1 Problem

Much of an organisation's data sits in relational databases, and getting answers out of them normally requires writing SQL. Administrative staff, faculty and management who need a number — how many students failed more than two subjects, which department has the highest average marks — usually have to ask someone who can write the query. Text-to-SQL systems remove that step: they translate a plain-English question into a query, run it and return the result.

Recent language models do this well on benchmarks. The difficulty is what happens when they are wrong. A text-to-SQL system answers every question, including the ones it gets wrong, and a wrong table looks just as authoritative as a correct one. For a user who cannot read SQL, that is the central risk: silently wrong answers. Most strong systems also depend on closed, hosted models, which means sending the database schema to a third party and accepting results that cannot be reproduced exactly.

1.2 Motivation

CodeS [1] is a family of open-source text-to-SQL models that addresses the second problem: its weights, code and evaluation setup are public, so it can run entirely on local hardware and its published numbers can be checked. It does not address the first. Like other systems, it returns one query per question with no indication of how likely that query is to be right.

This project starts from the observation that CodeS already computes the information needed to know when it is unsure. At inference time it produces four candidate queries with beam search and keeps one. If those candidates agree, the answer is more likely to be correct; if they disagree, the question is probably ambiguous or hard. Using that agreement to decide when *not* to answer — and asking the user which reading they meant instead — turns a model that is sometimes silently wrong into one that says when it does not know.

1.3 Objectives

Following the department's Phase-I procedure, the project had four objectives:

1. Reproduce the published results of CodeS within the department's tolerance of ±5–10%, and document every deviation from the authors' setup.
2. Establish which parts of the CodeS pipeline matter, through an ablation study.
3. Propose, implement and statistically test one enhancement: confidence-aware abstention.
4. Deliver the system as a working application rather than a notebook.

1.4 Contributions

- **A faithful reproduction** of fine-tuned CodeS-1B on the Spider benchmark [2]: 77.8% Execution Accuracy against the reported 77.9%, with an audited evaluation harness and a documented list of the remaining deviations.
- **An ablation study** of the CodeS database prompt and inference procedure, scored with the official evaluator.
- **Confidence-aware abstention** based on execution agreement between beam candidates, with a hypothesis fixed before the results were seen, significance tests, and a measured trade-off between the share of questions answered and the accuracy on them.
- **QueryGenie**, a desktop application for macOS and Windows that runs the evaluated pipeline locally, reports how many candidate queries agree, and asks for clarification when they do not.

1.5 Scope

CodeS is built by incrementally pre-training StarCoder [3] on SQL-centric data using eight A800 80 GB GPUs, which is far beyond a student budget. The project therefore reproduces the published *inference-time* results using the released fine-tuned checkpoint, which is the configuration behind the 77.9% figure. Pre-training and fine-tuning were not repeated.

1.6 Organisation of the report

Chapter 2 reviews the text-to-SQL literature. Chapter 3 describes the Spider benchmark, where and how it was obtained, and a preliminary analysis of its questions and of the reproduced model's accuracy on them. Chapter 4 presents the approach: the architecture, the reproduction methodology and results, the ablation study, the enhancement and its statistical evaluation, the application, and how to run the code. Chapter 5 concludes and outlines future work.

Chapter 2

Literature survey

The literature reviewed for this project falls into five groups: surveys, benchmark datasets, neural and pre-trained approaches, methods built on large language models (LLMs), and the presentation of query results. All papers were obtained from arXiv, as the department requires.

2.1 Surveys

Several recent surveys map the field. Two cover LLM-based text-to-SQL in depth [4, 5], and two further reviews examine LLM-based systems and how LLMs are employed for the task [6, 7]. Earlier surveys describe the concepts, methods and open directions of text-to-SQL parsing before the LLM era [8, 9]. Together they trace the shift from task-specific neural parsers to general-purpose language models prompted or fine-tuned for SQL generation.

2.2 Benchmark datasets

Spider [2] is the standard cross-domain benchmark: questions over many databases, with training, development and test sets that use disjoint databases, so a model must generalise to schemas it has not seen. BIRD [10] extends the setting to large-scale, database-grounded evaluation. Variants of Spider probe robustness: Spider-Syn replaces schema words with synonyms [11], Spider-DK requires domain knowledge [12], and Spider-Realistic removes explicit column mentions [13]. SParC [14] and CoSQL [15] move to multi-turn and conversational settings, CSpider [16] to Chinese questions, and NatSQL [17] proposes an intermediate representation that is easier to infer from natural language. Test-suite accuracy [18] evaluates a predicted query by running it against many generated variants of each database, which is stricter than a single execution. Across this work, Execution Accuracy has become the preferred metric over exact string match, because semantically equivalent queries can differ in form.

2.3 Neural and pre-trained approaches

Seq2SQL [19] generated structured queries with reinforcement learning on single-table data. For cross-domain parsing, RAT-SQL [20] introduced relation-aware schema encoding and linking, and LGESQL [21] used line-graph relations over the schema. Pre-training on tables and text was explored by TaBERT [22], GraPPa [23] and STAR [24]; BRIDGE [25] bridged textual and tabular data by adding matched cell values to the input — an idea CodeS keeps in its value retriever. PICARD [26] constrained auto-regressive decoding so that only valid SQL is produced, and RASAT [27] and T5-SR [28] built on pre-trained sequence-to-sequence models. RESDSQL [29], from the same group as CodeS and this project's fallback anchor, decouples schema linking from skeleton parsing and ranks schema items with a classifier.

Table 2.1: Results reported in the CodeS paper (its Table 5), Spider development set.

Model (fine-tuned on Spider)	**EX (%)**	**TS (%)**
CodeS-1B — reproduced in this project	77.9	72.2
CodeS-3B	83.4	78.1
CodeS-7B	85.4	80.3
CodeS-15B	84.9	79.4

2.4 Methods based on large language models

With large language models, zero-shot prompting became competitive [30], and Binder [31] bound language models to symbolic languages. DIN-SQL [32] decomposes the task into sub-problems and adds self-correction: when a query fails, the error is fed back to the model. DAIL-SQL [33] benchmarks prompt designs, MAC-SQL [34] coordinates several agents, and DBCopilot [35] routes questions across very large collections of databases. CHESS [36], E-SQL [37] and CHASE-SQL [38] improve schema linking, question enrichment and the selection among multiple candidate queries; one study asks whether schema linking is still needed with strong reasoning models [39]. Super-SQL [40] and DB-GPT-Hub [41] assess how ready these systems are and provide open benchmarking, and LLM routing has been studied to trade cost against quality [42].

CodeS [1], the paper reproduced here, takes a different route from most of this work: rather than prompting a large closed model, it builds open models of 1B to 15B parameters by incrementally pre-training StarCoder [3] on SQL-centric data and fine-tuning them, and it releases the weights. Its reported Spider development-set results for the fine-tuned models are summarised in Table 2.1.

2.5 Presenting results and knowing when to abstain

LIDA [43] generates visualisations of data automatically with language models; combining such result presentation with text-to-SQL remains little explored, and QueryGenie

chooses a chart for each result. Two further ideas underpin the enhancement. Self-consistency [44] samples several reasoning paths and trusts the answer they agree on. Selective classification [45] lets a model decline inputs on which it is not confident, and evaluates the trade-off between the share of inputs answered (coverage) and the accuracy on them.

2.6 Findings and the gap addressed

- Spider and BIRD are the accepted benchmarks, and Execution Accuracy is the preferred metric.
- The field has moved from graph-based encoders and constrained decoding to approaches based on large language models.
- Schema linking is repeatedly identified as the main bottleneck.
- Most of the strongest systems depend on closed, hosted models, which raises cost, privacy and reproducibility concerns; CodeS was proposed to address this with open weights.
- Self-correction and the selection among multiple candidates consistently help.
- What the reviewed systems do not do is report how sure they are. They answer every question with the same apparent confidence. This project uses the candidates CodeS already produces to measure agreement, abstains when agreement is low, and evaluates the result as an accuracy–coverage trade-off with significance tests.

Chapter 3

Data Collection and Preliminary Analysis

3.1 The Spider benchmark

The reproduction and the enhancement are evaluated on Spider [2], the benchmark on which CodeS reports its results. Spider is a cross-domain text-to-SQL dataset: each example is a natural-language question, a gold SQL query that answers it, and the SQLite database the query runs on. The training, development and test sets use disjoint databases, so a model is always evaluated on schemas it has never seen.

3.2 Data collection

The department permits data only from approved sources and explicitly bans Kaggle. Spider was therefore obtained from Hugging Face, which is on the approved list:

- **Questions and gold queries:** the Hugging Face dataset `xlangai/spider`,

Table 3.1: Spider splits used in this project.

Split	Questions	Databases
Training (Spider)	7,000	140
Training (other sources)	1,659	6
Development (evaluation)	1,034	20

maintained by the original authors' organisation, downloaded on 14 August 2026 by `scripts/download_spider.py`. The provenance (source, split sizes, library versions, time of download) is recorded in `results/dataset_provenance.json`. The older bare `spider` identifier no longer loads with current versions of the `datasets` library; `xlangai/spider` is the maintained copy of the same data.

- **Databases and schema file:** the SQLite databases and `tables.json` that the questions refer to, from the Hugging Face repository `HAL-9001/spider-databases` (`scripts/download_spider_dbs.py`).
- Kaggle was not used at any point.

3.3 Size and composition

Table 3.1 gives the size of each split. All results in this report are on the development set, as in the CodeS paper; the test set's gold queries were not used.

None of the 20 development databases occurs in either training split. The development databases are small: they have 2 to 11 tables (4.05 on average, 81 in total), 7 to 56 columns (22.05 on average, 441 in total) and 1 to 11 foreign keys (3.2 on average). The number of questions per database varies from 4 to 120, with a mean of 51.7.

Table 3.2: SQL features in the development-set gold queries (a query can have several).

Feature	Queries	Share
JOIN	408	39.5%
GROUP BY	277	26.8%
ORDER BY	237	22.9%
LIMIT	189	18.3%
Nested SELECT	83	8.0%
Set operation (UNION, INTERSECT, EXCEPT)	80	7.7%
HAVING	79	7.6%

3.4 Preliminary analysis

3.4.1 What the questions ask for

Table 3.2 counts SQL features in the 1,034 gold queries of the development set. Almost two in five questions need a join between tables, about one in twelve needs a nested query, and about as many a set operation. These are the constructs where a query can run without error and still return the wrong rows — a point that becomes central to the choice of enhancement in Chapter 4.

3.4.2 Difficulty

Spider assigns every query one of four difficulty levels according to its SQL components. Table 3.3 gives the number of development questions at each level, computed with the official evaluator's own classifier [18], together with the Execution Accuracy of the reproduced CodeS-1B model at that level. The accuracies are taken from the saved record of run T2-006 (Chapter 4), and they add up to its overall result: 804 of 1,034 questions correct.

Accuracy falls steadily with difficulty, from 90.3% on easy questions to 56.6% on extra-hard ones. Hard and extra-hard questions make up a third of the development set (340 of 1,034) but account for 133 of the 230 wrong answers. A single accuracy figure

Table 3.3: Development set by official difficulty level, and the reproduced CodeS-1B Execution Accuracy (run T2-006).

Difficulty	Questions	Correct	EX
Easy	248	224	90.3%
Medium	446	373	83.6%
Hard	174	113	64.9%
Extra hard	166	94	56.6%
All	**1,034**	**804**	**77.8%**

therefore hides very different reliability across questions, which is the motivation for reporting confidence per question rather than one number for the system.

3.5 Preparing the input for the model

CodeS does not see the raw database. For each question it builds a textual database prompt, exactly as in the authors' code:

- **Schema sequence:** every table with its columns, column types, primary keys, the foreign keys, and a few representative values per column read from the database.
- **Column descriptions:** human-readable names from Spider's `tables.json`, where they differ from the column name.
- **Matched contents:** cell values that match words in the question, found by the authors' value retriever (string similarity with `difflib` and `rapidfuzz`, score at least 0.9, at most 25 values per column).

The question follows the database prompt, and the beginning-of-sequence token is prepended. The longest prompt in the development set is 1,717 tokens, well inside the model's limit of 3,840 input tokens, so no prompt is ever truncated.

3.6 Data used by the application

The desktop application ships with a small academic database of its own — a `student` table (6 students) and a `result` table (18 marks) — which it builds on first start, and it can download the 20 Spider development databases above so that users can try the same databases the accuracy was measured on. Users can also open their own SQLite files or connect to PostgreSQL and MySQL servers. None of this application data is used in the evaluation.

Chapter 4

Proposed Approach

4.1 Architecture

Figure 4.1 shows the system as evaluated. The reproduced CodeS components build the database prompt, generate four candidate queries with beam search and select one. The enhancement executes all four candidates, measures how many return the same result, and decides whether to answer or to ask the user. The application wraps this pipeline in a desktop interface. The same code runs in the evaluation and in the application: the prompt, candidate generation and selection live in one module (`querygenie/pipeline.py`) that both use.

4.2 Reproduction methodology

4.2.1 The CodeS pipeline

Given a question Q and a database D, a text-to-SQL system produces a query S whose result answers Q. CodeS builds the database prompt described in Chapter 3 — schema

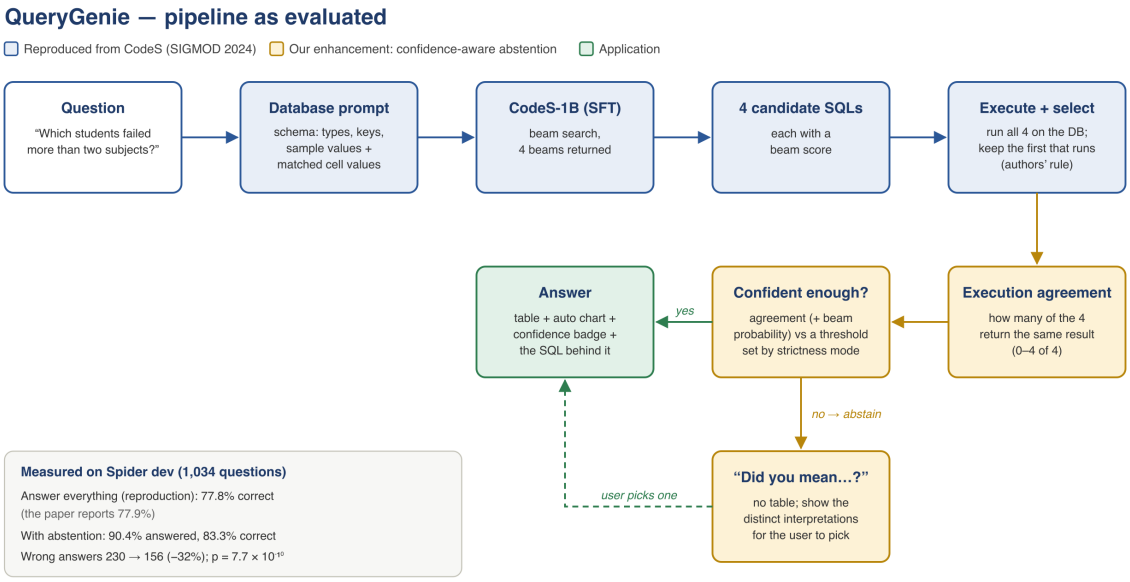


Figure 4.1: The pipeline as evaluated: reproduced CodeS components (blue), the enhancement (amber) and the application (green).

sequence, column descriptions and the matched cell values — and appends the question. A schema item classifier can prune the schema to its most relevant tables and columns. The model generates four candidates with beam search, and the first candidate that SQLite accepts is returned. Figure 4.2 summarises the pipeline as annotated from the paper.

4.2.2 Environment

Table 4.1 compares the authors' environment with ours. The reproduction first passed the department's Week-4 gate: the released model was loaded on a Colab T4 GPU, generated SQL and executed it against SQLite (run T1-001). The full evaluation followed.

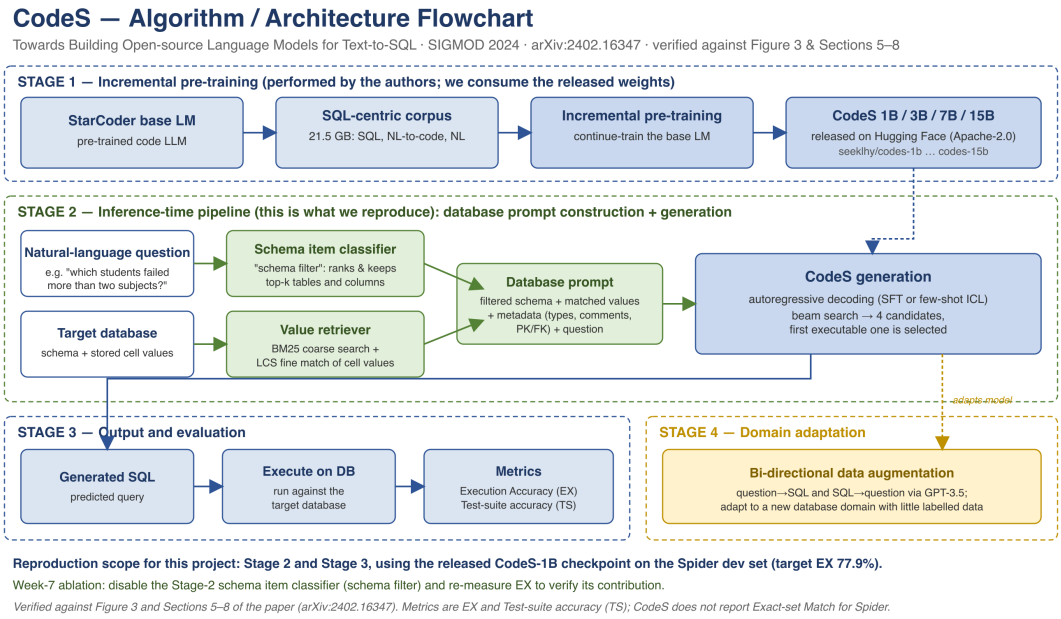


Figure 4.2: The CodeS pipeline, annotated from the paper (Week-3 deliverable).

Table 4.1: The authors' environment and ours.

	Authors (official repository)	**This project**
GPU	8 × NVIDIA A800, 80 GB	NVIDIA Tesla T4, 16 GB (Google Colab); Apple M4, 16 GB for development
Precision / batch	fp16, batch size 1 at inference	fp16, batch size 1
Python	3.8.5	Colab default (3.12.13 at the gate run)
PyTorch	1.13.1	2.11.0+cu128
transformers	4.28.1	5.15.0 (gate run), 5.16.1 (reported runs)
Dataset	Spider (Google Drive bundle)	Spider from Hugging Face (`xlangai/spider`, `HAL-9001/spider-databases`)

4.2.3 The reproduced pipeline

The evaluation harness (`research/colab_eval_tier2.py`) reproduces the authors' inference code, and the authors' database utilities are included unmodified (Apache-2.0):

- **Prompt:** schema sequence, matched contents and question, with the beginning-of-sequence token and the authors' left-truncation, exactly as in their `prepare_text2sql_prefix_sequence`.
- **Value retriever:** the authors' fine matcher applied to every column's distinct

values. The authors first narrow the candidates with a Lucene BM25 index; we scan all values instead (a documented deviation).

- **Decoding and selection:** beam search with four beams, all four returned and post-processed; the first candidate that passes SQLite's `EXPLAIN QUERY PLAN` is kept, otherwise the first beam — the rule in the authors' `text2sql_zero_shot.py`.
- **Scoring:** the official Spider test-suite evaluator (`evaluation.py --etype exec`) [18], which is what the authors call. Execution Accuracy (EX) counts a prediction as correct when it returns the same result as the gold query; the evaluator treats row order as significant only when the gold query sorts, tolerates reordered columns and ignores `DISTINCT` by default. Every prediction is saved so that any run can be re-scored.

4.2.4 Audit of the evaluation harness

After three runs gave 64.8%, 71.7% and 73.8%, the harness was compared line by line with the authors' code (`load_sft_dataset.py`, `prepare_sft_datasets.py`, `build_contents_index.py`, `text2sql_zero_shot.py` and `db_utils.py`), and every effect was measured on the full development set. Prompt assembly, decoding, candidate selection and post-processing matched, and the vendored code was identical to the authors'. Four discrepancies were found and fixed:

1. **Value-retriever bug.** The authors index every value as text, numbers included; our port kept only text values, so numbers in questions ("older than 20") were never matched. This changed the prompt for 128 of the 1,034 questions (12.4%).
2. **Scoring metric.** Our early metric compared sorted rows as strings: it ignored row order and did not accept reordered columns. On the same predictions it gives

73.4% where the official evaluator gives 77.8%, so the earlier runs were not on the paper's scale.

3. **Column descriptions.** The authors add human-readable column names from Spider's `tables.json`; 12 development-set columns are affected.

4. **Saved predictions.** Earlier runs did not save their SQL and could not be re-scored.

The official evaluator was checked by scoring the gold queries against themselves (1.000).

4.2.5 Run logging and determinism

Every run is logged with its seed (42), hardware, runtime and library versions in `results/reproduction_log.csv` and `results/Reproduction_Log.xlsx`, including runs that were later superseded. Beam search is deterministic: repeating a run returned exactly the same predictions (763 of 1,034 correct in T2-003 and its repeat; 759 of 1,034 on the simple metric in T2-005 and T2-006). Differences between runs therefore come from configuration changes, not chance.

4.3 Reproducibility results

4.3.1 Run history

Table 4.2 lists the runs on the development set. The reproduction reached its final figure in steps, and each step isolates a real effect: moving from a schema-only prompt to the authors' full database prompt raised the simple-metric score from 64.8% to 71.7%, the authors' candidate-selection rule raised it to 73.8%, and the audit showed that most of the remaining gap was the scoring metric rather than the model.

Table 4.2: Runs on the Spider development set (1,034 questions). "Simple" is the early sorted-rows metric; only "official" matches the paper.

Run	Configuration	Metric	EX	vs 77.9%
T1-001	Week-4 gate: codes-1b loads, generates and executes SQL (Colab T4)	liveness	passed	—
T2-001	schema and foreign keys only; first beam; Apple M4	simple	64.8%	not comparable
T2-002	full CodeS prompt; first beam; Colab T4	simple	71.7%	not comparable
T2-003	+ authors' candidate selection; Colab T4 (repeated identically)	simple	73.8%	not comparable
T2-005	audited harness (v2); Colab T4	official	77.8%	−0.1 pts
T2-006	fresh re-run of T2-005 (Week-10 baseline)	official	77.76% (804/1,034)	−0.14 pts

Table 4.3: Ablation study, official evaluator.

Component removed	EX	Change	Executable
— (full pipeline, T2-005)	77.8%	—	97.3%
Foreign-keys block	76.9%	−0.9	95.9%
Representative column values	76.4%	−1.4	98.1%
Candidate selection (first beam only)	76.0%	−1.8	94.1%
Value retriever (matched contents)	75.9%	−1.9	97.2%

4.3.2 Headline comparison

Scored with the paper's own evaluator, the reproduced CodeS-1B reaches **77.8% Execution Accuracy against the reported 77.9%**: a difference of 0.1 points, well inside the department's tolerance. The fresh Week-10 re-run gave the identical result (804 of 1,034 correct). 97.3% of the predictions execute.

Test-suite accuracy (paper: 72.2%) requires the distilled test-suite databases, which are distributed on Google Drive and were not obtained; our test-suite accuracy is **TBD**. The saved predictions of T2-006 can be scored for it without re-running the model.

4.3.3 Ablation study

Each component of the pipeline was disabled in turn and the full development set re-evaluated with the official evaluator (runs AB-103 to AB-106, Colab T4). Table 4.3 and Figure 4.3 give the results.

Every component contributes. The value retriever matters most, and the way it mat-

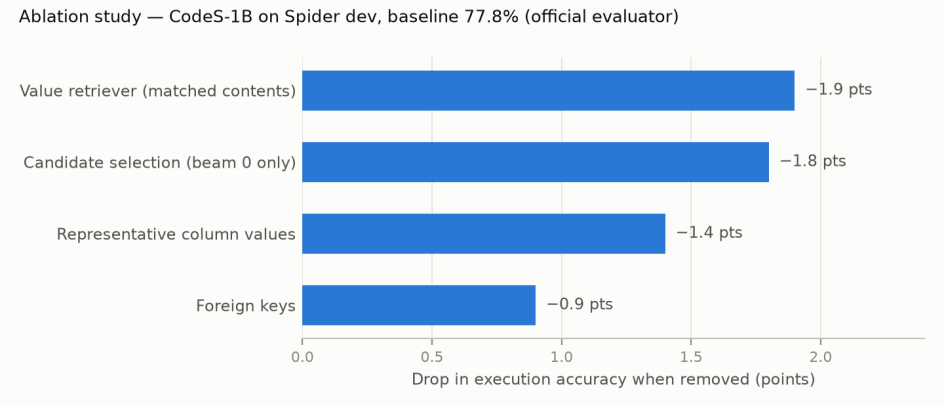


Figure 4.3: Drop in Execution Accuracy when each component is removed.

ters is telling: without it almost as many queries still execute (97.2%) — the model writes runnable SQL with the wrong literal values, so the answers are silently wrong. Candidate selection works differently: it is the ablation that reduces executability most (94.1%), because it is what replaces a broken first beam with a working alternative. Without the foreign keys the model guesses join conditions, which also shows up as lower executability. These differences are 9 to 20 questions each and were not tested for significance individually.

4.4 Enhancement: confidence-aware abstention

4.4.1 Choice and justification

The department allows one enhancement, chosen from defined tiers and approved by the guide. The Week-1 abstract had named execution-guided self-correction, the strategy of DIN-SQL [32]. The reproduction showed that 97.3% of predictions already execute, so repairing failed queries could recover at most 2.7 points and was unlikely to produce a significant gain. The ablation and the difficulty analysis of Chapter 3 showed instead that the dominant error is a query that runs but returns the wrong rows. We therefore proposed confidence-aware abstention (Tier 4, a novel combination), keeping

self-correction and automatic charts as application features.

That expectation was later measured directly (`research/repair_eval.py`). Keeping the four candidates of run T2-006 fixed, every failed candidate — 377 queries in 211 questions — was sent back to CodeS-1B once with its database error. Only 61 of the repaired queries ran, and one answer became correct (none became wrong): Execution Accuracy moved from 77.76% to 77.85%, and the tested abstention setting from 90.43% coverage at 83.32% to 90.52% at 83.23%. CodeS was fine-tuned on question–SQL pairs and never learned to use error feedback, which DIN-SQL's self-correction relies on. Self-correction therefore remains an application feature, on by default and labelled as such, and is not part of the evaluated results.

4.4.2 Method

The pipeline already produces four beam candidates per question. All four are executed. The confidence of the returned answer is the share of candidates whose result equals it — execution agreement, from 0 to 4 of 4, and zero when the returned query does not run — with the beam probability of the returned candidate (the softmax of the four beam scores) used only to break ties. The idea combines self-consistency [44] with selective classification [45]. When confidence is below a threshold the system abstains: it shows no table and offers the distinct results of the candidates as interpretations for the user to choose from. No retraining is involved. Algorithm 1 states the procedure.

4.4.3 Hypothesis and protocol

The hypothesis and protocol were fixed before the results were seen.

- **H1:** at the same coverage, accuracy on answered questions is higher with agreement-based abstention than with random abstention, and higher than the 77.8% base-

Algorithm 1: Confidence-aware abstention on top of CodeS

Input: question Q, database D, threshold τ
Output: an answer, or the readings for the user to choose from
$P \leftarrow$ CodeS database prompt for (Q, D);
$(s_1, \ldots, s_4), (b_1, \ldots, b_4) \leftarrow$ four beam candidates and their scores;
for $i \leftarrow 1$ **to** 4 **do**
  $r_i \leftarrow$ result of executing s_i on D (read-only; empty if it fails);
end
$k \leftarrow$ first i whose query passes `EXPLAIN QUERY PLAN`, else 1;
$a \leftarrow \frac{1}{4}\,|\{\, i : r_i = r_k \,\}|$ if s_k runs, else 0;
$p \leftarrow e^{b_k} / \sum_j e^{b_j}$;
$c \leftarrow a + 0.01\,p$;
if $c \geq \tau$ **then**
  return result r_k with confidence a;
else
  return the distinct results among $r_1, \ldots, r_4$ as readings;
end

line. **H0:** no difference.

- **Protocol:** 10 random 50/50 splits of the development set. On the calibration half the threshold is set for at least 90% coverage, then applied to the test half. The paired quantity is the accuracy on answered test questions minus the expected accuracy of random abstention at the same coverage, which is the test half's full-coverage accuracy. One-sided paired t-test and Wilcoxon signed-rank test, $\alpha = 0.05$.

- **Data:** one run (T2-006) recording, for every question, all four candidates with their beam scores, execution results and official correctness, checked to add up to the evaluator's total. The original and the enhanced systems are therefore compared on exactly the same predictions.

The threshold used by the application, $\tau = 0.2526$, is the median of the 10 calibration-half thresholds. In words: abstain when no candidate executes, or when only 1 of 4

Table 4.4: Original against enhanced on the same 1,034 questions.

System	Answered	Accuracy on answered	Wrong answers
CodeS-1B, paper (Table 5)	1,034 (100%)	77.9%	—
Original — CodeS-1B reproduced	1,034 (100%)	77.76%	230
Enhanced — tested setting	935 (90.4%)	83.32%	156
Enhanced — 3 of 4 agree	649 (62.8%)	86.13%	90
Enhanced — all 4 agree	412 (39.8%)	88.35%	48

Table 4.5: Significance tests over 10 calibration/test splits (mean coverage 90.5%) and a 5,000-sample bootstrap over questions.

Signal	Mean gain	Paired t-test (df 9)	Wilcoxon	Bootstrap 95% CI
Execution agreement	+5.46 pts (SD 0.71)	$t = 24.44$, $p = 7.7 \times 10^{-10}$	$W = 55$, $p = 9.8 \times 10^{-4}$	$[+4.51, +6.71]$
Beam probability (comparison)	+3.08 pts (SD 0.90)	$t = 10.81$, $p = 9.3 \times 10^{-7}$	$W = 55$, $p = 9.8 \times 10^{-4}$	$[+1.97, +3.94]$

candidates supports the answer and its beam probability is below about 0.26.

4.4.4 Results

Table 4.4 compares the original and enhanced systems on the same 1,034 questions, and Table 4.5 gives the significance tests.

H0 is rejected. All 10 splits showed a positive gain ($W = 55$ is the maximum, and 9.8×10^{-4} the smallest p-value the Wilcoxon test can give with 10 pairs). Because the splits reuse the same questions, the questions themselves were also resampled 5,000 times; no resample showed a gain of zero or less. At the tested setting the system declines 99 questions, 74 of which it would have answered wrongly and 25 correctly: wrong answers fall by 32% at the cost of withholding 3.1% of the correct ones.

Figure 4.4 shows the whole accuracy–coverage curve, and Table 4.6 shows that the signal is calibrated: accuracy rises steadily with agreement.

4.4.5 Limits of the enhancement

- **Beam probability is the better signal when abstaining heavily.** Execution agreement is ahead from 100% down to about 74% coverage — the region where

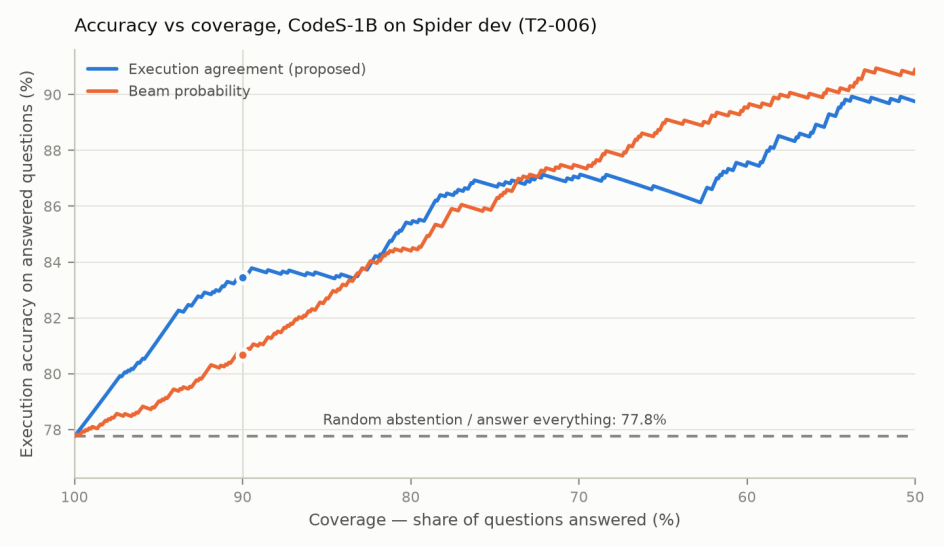


Figure 4.4: Accuracy on answered questions against coverage. The dashed line is the original system (and random abstention); dots mark 90% coverage.

Table 4.6: Accuracy by the number of candidates agreeing with the answer.

Candidates agreeing with the answer	**Questions**	**Accuracy**
None (selected query does not run)	28	0.0%
1 of 4	132	56.8%
2 of 4	225	75.6%
3 of 4	237	82.3%
4 of 4	412	88.3%

a usable assistant operates and where the proposal set its operating point — but below 74% beam probability is ahead, and over the whole curve it has the lower area under the risk–coverage curve (0.091 against 0.103; random 0.222, perfect ranking 0.027).

- **Agreement is not proof.** 11.7% of the answers on which all four candidates agree are still wrong; abstention reduces errors but does not remove them.
- **One model, one benchmark.** The threshold was calibrated on Spider with CodeS-1B; its behaviour with other models, and on real institutional databases, is not measured.

- **Statistical independence.** The 10 splits reuse the same questions, so their p-values are optimistic; the question-level bootstrap is the check that does not depend on this.

4.5 The application

The enhancement is delivered in QueryGenie, a desktop application for macOS and Windows, which meets the department's requirement for a full-stack deliverable rather than a notebook. On first start a setup assistant checks the computer and downloads the model (`seeklhy/codes-1b-spider`, about 4.5 GB) and, optionally, the 20 Spider development databases. It then runs exactly the evaluated pipeline, locally.

- **Chats.** Questions are asked in chats, one database at a time. Every answer shows how many of the four candidate queries agree (Figure 4.5), the results table, an automatically chosen chart, the SQL behind the table (editable, and re-runnable read-only) and all four candidates.
- **Abstention.** When the candidates disagree beyond the chosen strictness setting, the application withholds the table and shows the distinct readings for the user to choose from (Figure 4.6). The four strictness settings display their measured coverage and accuracy from Table 4.4.
- **Schema map.** The whole database is drawn as a tree of tables, columns, keys and links; tapping a table or column offers questions about it (Figure 4.7).
- **Databases.** Besides the sample and Spider databases, users can open their own SQLite files or connect to PostgreSQL and MySQL servers. Every query is checked to be a single read-only statement and runs inside a read-only transaction with a timeout and a row limit. The model writes SQLite-style SQL, which is translated

to the server's dialect: on the Spider databases loaded into PostgreSQL, translated queries return the same answer as on SQLite for 94.4% of gold queries and 93.7% of the model's predictions. Accuracy and confidence were calibrated on SQLite only.

- **Other models.** The evaluated CodeS-1B model is the default. The user can also choose the earlier prototype's CodeS variants, any model run by Ollama, an OpenAI-compatible server, or cloud services (Figure 4.8). They receive the same four-candidate confidence check, but their accuracy was not measured, and the application labels their confidence as not calibrated. Cloud services receive the schema, a few example values and the question, never result rows, and only after the user confirms.

By default everything runs on the user's computer on open weights, so no schema or data leaves the machine.

A qualitative check on the bundled academic database shows both sides of the behaviour (illustrative, not a metric). With CodeS-1B the flagship question "Which students failed more than two subjects?" gets a wrong first answer, but the candidates split two against two, the answer is marked low confidence, the correct reading is offered as an alternative, and the strict setting abstains (Figure 4.6). A system that says when it is unsure is only as useful as that signal is honest, which is why the application shows its measured operating points rather than a single headline number.

4.6 Differences from the authors' setup

The reproduced accuracy is within 0.1 points of the paper, so the remaining deviations appear to have little effect on this benchmark. They are recorded in Table 4.7 for

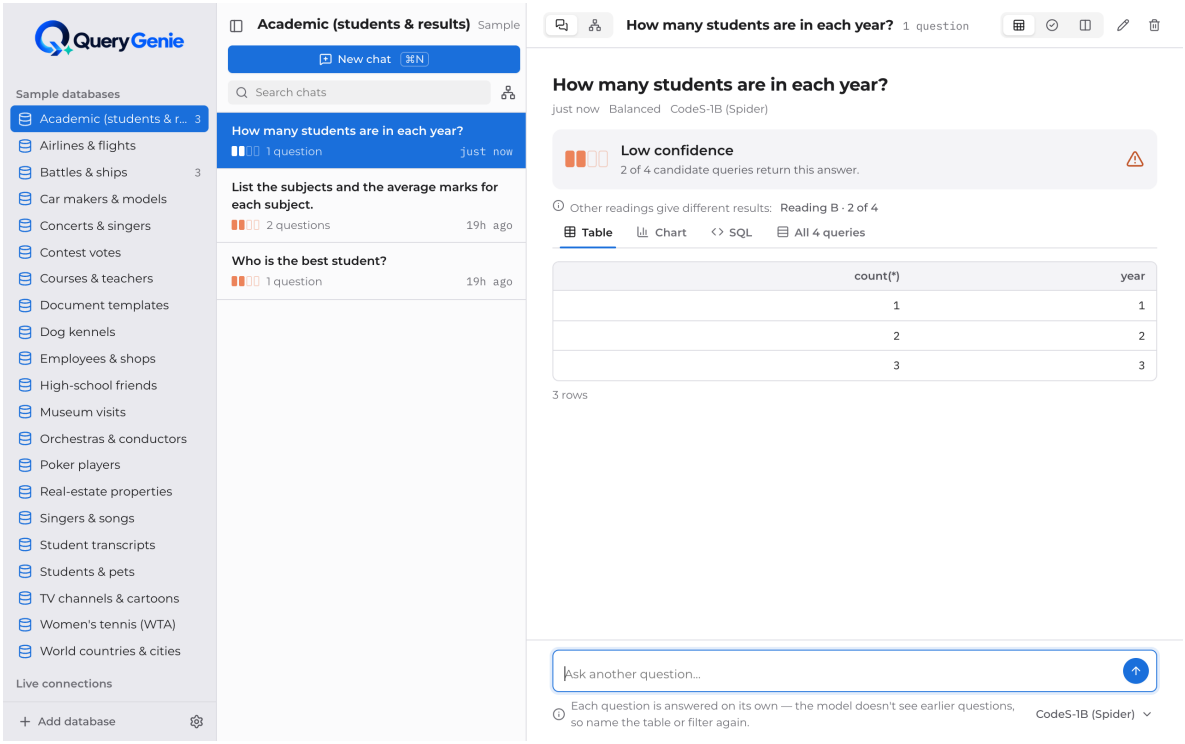


Figure 4.5: An answer with its confidence: two of the four candidate queries agree, so the answer is shown as low confidence, with the other reading offered.

Table 4.7: Remaining deviations from the authors' setup.

Deviation	Cause	Observed or expected effect
Newer PyTorch and transformers	The pinned stack does not build on current Python	The authors note newer versions shift inference slightly; no measurable gap remains
Distinct-value scan instead of Lucene BM25	Avoids the Java / pyserini dependency	Superset of the authors' candidates; matched values may differ in number and order
Schema item classifier not applied	Checkpoints on rate-limited Google Drive	Full schema passed; Spider databases are small
Tesla T4 instead of A800	Available hardware	Same precision (fp16) and batch size (1) as the authors' inference
EX only; test-suite accuracy TBD	Test-suite databases not obtained	Test-suite accuracy (paper: 72.2%) not yet reported
Repository uses DeepSpeed ZeRO-2; paper states ZeRO-3	Found while annotating the paper	Affects pre-training only, which was not reproduced

completeness.

The largest lesson of the reproduction is about measurement rather than modelling. For a month our numbers sat 4 to 13 points below the paper, and the gap was blamed in turn on the prompt, the value retriever and the schema filter. The audit showed that about 4.4 points of it were our own scoring. Reproducing a result means reproducing the evaluation, not only the model.

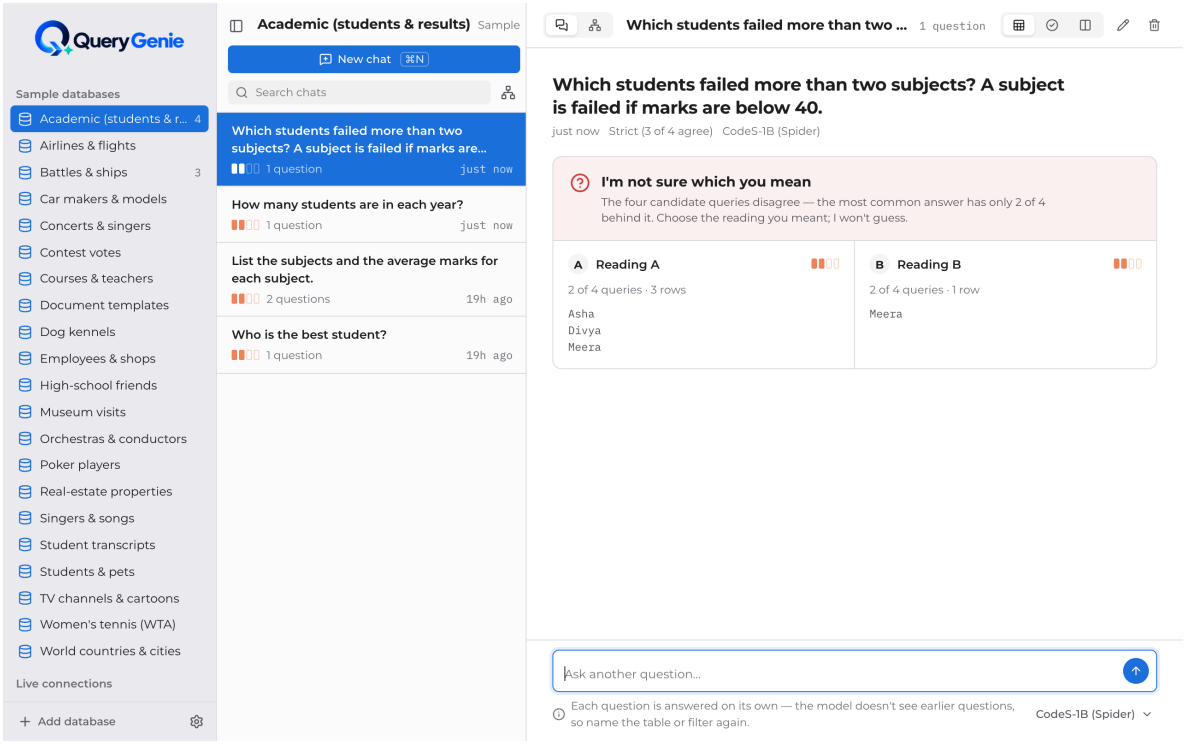


Figure 4.6: Abstention under the strict setting: the candidates split two against two on the flagship question, so the application asks which reading was meant instead of guessing.

4.7 Source code and how to run it

All code, results and documents are in the project repository on GitHub, `https://github.com/jayakanthh/QueryGenie`, with the supervisor added as a collaborator. The code is released under the Apache 2.0 licence. The repository is organised as follows:

- `querygenie/` — the evaluated core: the CodeS pipeline, the model wrapper, the abstention logic, the database catalogue and the connectors; the authors' utilities are included unmodified.
- `desktop/` — the desktop application (native window, interface, first-run setup, packaging).

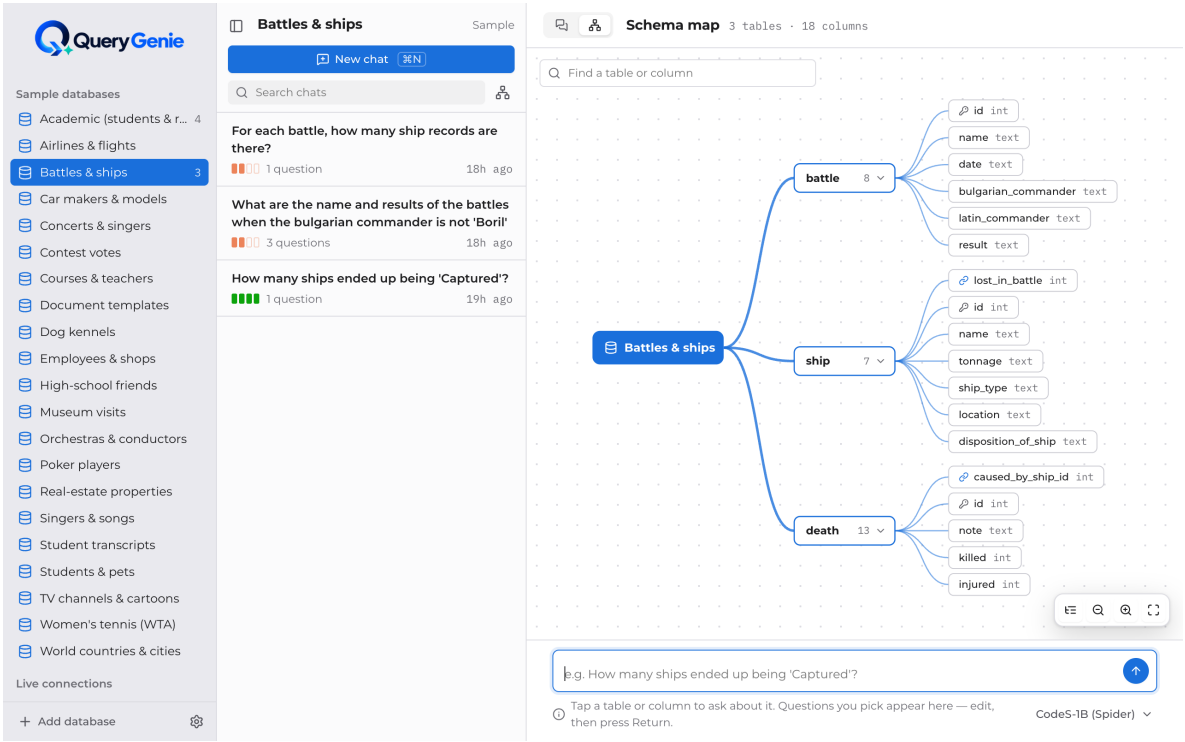


Figure 4.7: The schema map: tables, columns, primary keys and foreign keys of a Spider database.

- `research/` — the Colab evaluation harness, the abstention analysis, the dataset analysis of Chapter 3 and the notebooks.
- `results/` — the run log, the harness audit, the ablation, abstention and comparison results, the figures, and the four-candidate record of run T2-006.
- `legacy/` — the earlier web prototype, kept for the record; `tests/` — model-free tests.

Reproduce the results (GPU, about one hour). Open `research/notebooks/02_tier2_colab_eval.ipynb` in Google Colab with a T4 runtime and run the Week-10 cell. It downloads Spider and the model from Hugging Face, evaluates all 1,034 questions with the official evaluator, and saves the predictions and the four-candidate record.

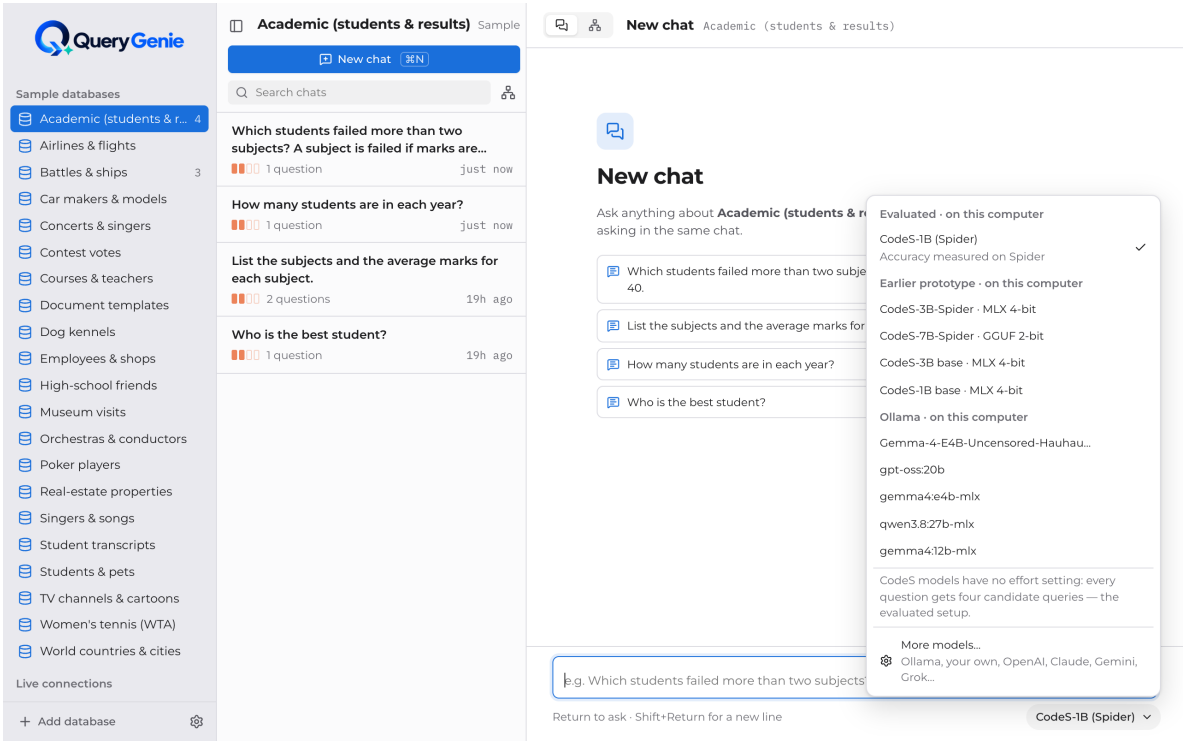


Figure 4.8: The model picker below the question box: the evaluated model, the earlier prototype's models and local Ollama models.

Reproduce the enhancement analysis (CPU, seconds).

```
python research/abstention.py results/dump_T2-006.jsonl
```

Listing 4.1: Significance tests, operating points and the accuracy–coverage figure

Run the application.

```
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
python desktop/main.py
```

Listing 4.2: Running QueryGenie from source

Tests (no model needed). `python -m pytest tests/ -q`.

Chapter 5

Conclusion and Future Scope of work

5.1 Conclusion

This project reproduced CodeS [1], an open-source text-to-SQL model, and extended it so that it says when it is unsure.

The fine-tuned CodeS-1B model was reproduced to within 0.1 points of its published Spider result: 77.8% Execution Accuracy against 77.9%, scored with the official evaluator, on hardware far smaller than the authors'. Getting there required an audit of our own evaluation harness, which traced most of an apparent 4–13 point gap to the scoring metric and a value-retriever bug rather than to the model. An ablation study showed that every part of the CodeS database prompt contributes 0.9 to 1.9 points, with the value retriever mattering most.

On top of the reproduced model, confidence-aware abstention uses the agreement between the four candidate queries CodeS already produces. At the pre-registered operat-

ing point it answers 90.4% of questions and is correct on 83.3% of them, against 77.8% when answering everything, and it removes about a third of the wrong answers (230 to 156). The improvement is significant under a paired t-test, a Wilcoxon signed-rank test and a question-level bootstrap. Accuracy rises steadily with the number of agreeing candidates, from 56.8% when only one agrees to 88.3% when all four do, so the signal is calibrated.

The system is delivered as QueryGenie, a desktop application that runs the evaluated pipeline locally, shows how sure it is on every answer, and asks which reading the user meant when its candidates disagree.

5.2 Future scope of work

1. **Test-suite accuracy.** Score the saved predictions of run T2-006 on the distilled test-suite databases, to report the stricter metric the paper also gives (72.2%).

2. **Larger models.** Evaluate CodeS-3B (paper: 83.4%) and calibrate the abstention threshold for it rather than reusing the 1B threshold.

3. **A combined signal.** Combine execution agreement and beam probability, which lead at different coverages, into one confidence score tuned only on calibration data.

4. **The benchmark-to-reality gap.** Measure accuracy and abstention on a real institutional schema, where names, values and questions are messier than in Spider.

5. **More databases.** Extend the connectors to further SQL engines and data warehouses, and to document stores through a local SQL snapshot, measuring translation accuracy for each as was done for PostgreSQL.

6. **Other models.** Measure the accuracy and calibration of the other models the application can use, so that their confidence can be presented with the same evidence as CodeS-1B's.

Bibliography

[1] H. Li, J. Zhang, H. Liu, J. Fan, X. Zhang, J. Zhu, R. Wei, H. Pan, C. Li, and H. Chen, "CodeS: Towards Building Open-source Language Models for Text-to-SQL," *arXiv preprint arXiv:2402.16347*, 2024, sIGMOD 2024. [Online]. Available: https://arxiv.org/abs/2402.16347

[2] T. Yu, R. Zhang, K. Yang, M. Yasunaga, D. Wang, Z. Li, J. Ma, I. Li, Q. Yao, S. Roman, Z. Zhang, and D. Radev, "Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task," *arXiv preprint arXiv:1809.08887*, 2018, eMNLP 2018. [Online]. Available: https://arxiv.org/abs/1809.08887

[3] R. Li, L. B. Allal, Y. Zi, N. Muennighoff, D. Kocetkov, C. Mou, M. Marone, C. Akiki, J. Li, J. Chim, Q. Liu, E. Zheltonozhskii *et al.*, "StarCoder: may the source be with you!" *arXiv preprint arXiv:2305.06161*, 2023, arXiv 2023. [Online]. Available: https://arxiv.org/abs/2305.06161

[4] Z. Hong, Z. Yuan, Q. Zhang, H. Chen, J. Dong, F. Huang, and X. Huang, "Next-Generation Database Interfaces: A Survey of LLM-based Text-to-SQL," *arXiv preprint arXiv:2406.08426*, 2024, tKDE 2025. [Online]. Available: https://arxiv.org/abs/2406.08426

[5] X. Liu, S. Shen, B. Li, P. Ma, R. Jiang, Y. Zhang, J. Fan, G. Li, N. Tang, and Y. Luo, "A Survey of Text-to-SQL in the Era of LLMs: Where are we, and where are we going?" *arXiv preprint arXiv:2408.05109*, 2024, tKDE 2025. [Online]. Available: https://arxiv.org/abs/2408.05109

[6] A. Mohammadjafari, A. S. Maida, and R. Gottumukkala, "From Natural Language to SQL: Review of LLM-based Text-to-SQL Systems," *arXiv*

preprint arXiv:2410.01066, 2024, arXiv 2024. [Online]. Available: https://arxiv.org/abs/2410.01066

[7] L. Shi, Z. Tang, N. Zhang, X. Zhang, and Z. Yang, "A Survey on Employing Large Language Models for Text-to-SQL Tasks," *arXiv preprint arXiv:2407.15186*, 2024, arXiv 2024. [Online]. Available: https://arxiv.org/abs/2407.15186

[8] B. Qin, B. Hui, L. Wang, M. Yang, J. Li, B. Li, R. Geng, R. Cao, J. Sun, L. Si, F. Huang, and Y. Li, "A Survey on Text-to-SQL Parsing: Concepts, Methods, and Future Directions," *arXiv preprint arXiv:2208.13629*, 2022, tKDE 2022. [Online]. Available: https://arxiv.org/abs/2208.13629

[9] N. Deng, Y. Chen, and Y. Zhang, "Recent Advances in Text-to-SQL: A Survey of What We Have and What We Expect," *arXiv preprint arXiv:2208.10099*, 2022, cOLING 2022. [Online]. Available: https://arxiv.org/abs/2208.10099

[10] J. Li, B. Hui, G. Qu, J. Yang, B. Li, B. Li, B. Wang, B. Qin, R. Cao, R. Geng, N. Huo, X. Zhou *et al.*, "Can LLM Already Serve as A Database Interface? A BIg Bench for Large-Scale Database Grounded Text-to-SQLs," *arXiv preprint arXiv:2305.03111*, 2023, neurIPS 2023. [Online]. Available: https://arxiv.org/abs/2305.03111

[11] Y. Gan, X. Chen, Q. Huang, M. Purver, J. R. Woodward, J. Xie, and P. Huang, "Towards Robustness of Text-to-SQL Models against Synonym Substitution," *arXiv preprint arXiv:2106.01065*, 2021, aCL 2021. [Online]. Available: https://arxiv.org/abs/2106.01065

[12] Y. Gan, X. Chen, and M. Purver, "Exploring Underexplored Limitations of Cross-Domain Text-to-SQL Generalization," *arXiv preprint arXiv:2109.05157*, 2021, eMNLP 2021. [Online]. Available: https://arxiv.org/abs/2109.05157

[13] X. Deng, A. H. Awadallah, C. Meek, O. Polozov, H. Sun, and M. Richardson, "Structure-Grounded Pretraining for Text-to-SQL," *arXiv preprint arXiv:2010.12773*, 2020, nAACL 2021. [Online]. Available: https://arxiv.org/abs/2010.12773

[14] T. Yu, R. Zhang, M. Yasunaga, Y. C. Tan, X. V. Lin, S. Li, H. Er, I. Li, B. Pang, T. Chen, E. Ji, S. Dixit *et al.*, "SParC: Cross-Domain Semantic Parsing in

Context," *arXiv preprint arXiv:1906.02285*, 2019, aCL 2019. [Online]. Available: https://arxiv.org/abs/1906.02285

[15] T. Yu, R. Zhang, H. Y. Er, S. Li, E. Xue, B. Pang, X. V. Lin, Y. C. Tan, T. Shi, Z. Li, Y. Jiang, M. Yasunaga *et al.*, "CoSQL: A Conversational Text-to-SQL Challenge Towards Cross-Domain Natural Language Interfaces to Databases," *arXiv preprint arXiv:1909.05378*, 2019, eMNLP 2019. [Online]. Available: https://arxiv.org/abs/1909.05378

[16] Q. Min, Y. Shi, and Y. Zhang, "A Pilot Study for Chinese SQL Semantic Parsing," *arXiv preprint arXiv:1909.13293*, 2019, eMNLP 2019. [Online]. Available: https://arxiv.org/abs/1909.13293

[17] Y. Gan, X. Chen, J. Xie, M. Purver, J. R. Woodward, J. Drake, and Q. Zhang, "Natural SQL: Making SQL Easier to Infer from Natural Language Specifications," *arXiv preprint arXiv:2109.05153*, 2021, eMNLP 2021. [Online]. Available: https://arxiv.org/abs/2109.05153

[18] R. Zhong, T. Yu, and D. Klein, "Semantic Evaluation for Text-to-SQL with Distilled Test Suites," *arXiv preprint arXiv:2010.02840*, 2020, eMNLP 2020. [Online]. Available: https://arxiv.org/abs/2010.02840

[19] V. Zhong, C. Xiong, and R. Socher, "Seq2SQL: Generating Structured Queries from Natural Language using Reinforcement Learning," *arXiv preprint arXiv:1709.00103*, 2017, arXiv 2017. [Online]. Available: https://arxiv.org/abs/1709.00103

[20] B. Wang, R. Shin, X. Liu, O. Polozov, and M. Richardson, "RAT-SQL: Relation-Aware Schema Encoding and Linking for Text-to-SQL Parsers," *arXiv preprint arXiv:1911.04942*, 2019, aCL 2020. [Online]. Available: https://arxiv.org/abs/1911.04942

[21] R. Cao, L. Chen, Z. Chen, Y. Zhao, S. Zhu, and K. Yu, "LGESQL: Line Graph Enhanced Text-to-SQL Model with Mixed Local and Non-Local Relations," *arXiv preprint arXiv:2106.01093*, 2021, aCL 2021. [Online]. Available: https://arxiv.org/abs/2106.01093

[22] P. Yin, G. Neubig, W. tau Yih, and S. Riedel, "TaBERT: Pretraining for Joint Understanding of Textual and Tabular Data," *arXiv preprint arXiv:2005.08314*,

2020, aCL 2020. [Online]. Available: https://arxiv.org/abs/2005.08314

[23] T. Yu, C.-S. Wu, X. V. Lin, B. Wang, Y. C. Tan, X. Yang, D. Radev, R. Socher, and C. Xiong, "GraPPa: Grammar-Augmented Pre-Training for Table Semantic Parsing," *arXiv preprint arXiv:2009.13845*, 2020, iCLR 2021. [Online]. Available: https://arxiv.org/abs/2009.13845

[24] Z. Cai, X. Li, B. Hui, M. Yang, B. Li, B. Li, Z. Cao, W. Li, F. Huang, L. Si, and Y. Li, "STAR: SQL Guided Pre-Training for Context-dependent Text-to-SQL Parsing," *arXiv preprint arXiv:2210.11888*, 2022, eMNLP 2022. [Online]. Available: https://arxiv.org/abs/2210.11888

[25] X. V. Lin, R. Socher, and C. Xiong, "Bridging Textual and Tabular Data for Cross-Domain Text-to-SQL Semantic Parsing," *arXiv preprint arXiv:2012.12627*, 2020, eMNLP 2020. [Online]. Available: https://arxiv.org/abs/2012.12627

[26] T. Scholak, N. Schucher, and D. Bahdanau, "PICARD: Parsing Incrementally for Constrained Auto-Regressive Decoding from Language Models," *arXiv preprint arXiv:2109.05093*, 2021, eMNLP 2021. [Online]. Available: https://arxiv.org/abs/2109.05093

[27] J. Qi, J. Tang, Z. He, X. Wan, Y. Cheng, C. Zhou, X. Wang, Q. Zhang, and Z. Lin, "RASAT: Integrating Relational Structures into Pretrained Seq2Seq Model for Text-to-SQL," *arXiv preprint arXiv:2205.06983*, 2022, eMNLP 2022. [Online]. Available: https://arxiv.org/abs/2205.06983

[28] Y. Li, Z. Su, Y. Li, H. Zhang, S. Wang, W. Wu, and Y. Zhang, "T5-SR: A Unified Seq-to-Seq Decoding Strategy for Semantic Parsing," *arXiv preprint arXiv:2306.08368*, 2023, iCASSP 2023. [Online]. Available: https://arxiv.org/abs/2306.08368

[29] H. Li, J. Zhang, C. Li, and H. Chen, "RESDSQL: Decoupling Schema Linking and Skeleton Parsing for Text-to-SQL," *arXiv preprint arXiv:2302.05965*, 2023, aAAI 2023. [Online]. Available: https://arxiv.org/abs/2302.05965

[30] A. Liu, X. Hu, L. Wen, and P. S. Yu, "A comprehensive evaluation of ChatGPT's zero-shot Text-to-SQL capability," *arXiv preprint arXiv:2303.13547*, 2023, arXiv 2023. [Online]. Available: https://arxiv.org/abs/2303.13547

[31] Z. Cheng, T. Xie, P. Shi, C. Li, R. Nadkarni, Y. Hu, C. Xiong, D. Radev, M. Ostendorf, L. Zettlemoyer, N. A. Smith, and T. Yu, "Binding Language Models in Symbolic Languages," *arXiv preprint arXiv:2210.02875*, 2022, iCLR 2023. [Online]. Available: https://arxiv.org/abs/2210.02875

[32] M. Pourreza and D. Rafiei, "DIN-SQL: Decomposed In-Context Learning of Text-to-SQL with Self-Correction," *arXiv preprint arXiv:2304.11015*, 2023, neurIPS 2023. [Online]. Available: https://arxiv.org/abs/2304.11015

[33] D. Gao, H. Wang, Y. Li, X. Sun, Y. Qian, B. Ding, and J. Zhou, "Text-to-SQL Empowered by Large Language Models: A Benchmark Evaluation," *arXiv preprint arXiv:2308.15363*, 2023, vLDB 2024. [Online]. Available: https://arxiv.org/abs/2308.15363

[34] B. Wang, C. Ren, J. Yang, X. Liang, J. Bai, L. Chai, Z. Yan, Q.-W. Zhang, D. Yin, X. Sun, and Z. Li, "MAC-SQL: A Multi-Agent Collaborative Framework for Text-to-SQL," *arXiv preprint arXiv:2312.11242*, 2023, aCL 2025. [Online]. Available: https://arxiv.org/abs/2312.11242

[35] T. Wang, X. Chen, H. Lin, X. Han, L. Sun, H. Wang, and Z. Zeng, "DBCopilot: Natural Language Querying over Massive Databases via Schema Routing," *arXiv preprint arXiv:2312.03463*, 2023, eDBT 2025. [Online]. Available: https://arxiv.org/abs/2312.03463

[36] S. Talaei, M. Pourreza, Y.-C. Chang, A. Mirhoseini, and A. Saberi, "CHESS: Contextual Harnessing for Efficient SQL Synthesis," *arXiv preprint arXiv:2405.16755*, 2024, arXiv 2024. [Online]. Available: https://arxiv.org/abs/2405.16755

[37] H. A. Caferoğlu and Ö. Ulusoy, "E-SQL: Direct Schema Linking via Question Enrichment in Text-to-SQL," *arXiv preprint arXiv:2409.16751*, 2024, arXiv 2024. [Online]. Available: https://arxiv.org/abs/2409.16751

[38] M. Pourreza, H. Li, R. Sun, Y. Chung, S. Talaei, G. T. Kakkar, Y. Gan, A. Saberi, F. Ozcan, and S. O. Arik, "CHASE-SQL: Multi-Path Reasoning and Preference Optimized Candidate Selection in Text-to-SQL," *arXiv preprint arXiv:2410.01943*, 2024, arXiv 2024. [Online]. Available: https://arxiv.org/abs/2410.01943

[39] K. Maamari, F. Abubaker, D. Jaroslawicz, and A. Mhedhbi, "The Death of Schema Linking? Text-to-SQL in the Age of Well-Reasoned Language Models," *arXiv preprint arXiv:2408.07702*, 2024, neurIPS 2024. [Online]. Available: https://arxiv.org/abs/2408.07702

[40] B. Li, Y. Luo, C. Chai, G. Li, and N. Tang, "The Dawn of Natural Language to SQL: Are We Fully Ready?" *arXiv preprint arXiv:2406.01265*, 2024, vLDB 2024. [Online]. Available: https://arxiv.org/abs/2406.01265

[41] F. Zhou, S. Xue, D. Qi, W. Shi, W. Zhao, G. Wei, H. Zhang, C. Jiang, G. Jiang, Z. Chu, and F. Chen, "DB-GPT-Hub: Towards Open Benchmarking Text-to-SQL Empowered by Large Language Models," *arXiv preprint arXiv:2406.11434*, 2024, arXiv 2024. [Online]. Available: https://arxiv.org/abs/2406.11434

[42] M. Malekpour, N. Shaheen, F. Khomh, and A. Mhedhbi, "Towards Optimizing SQL Generation via LLM Routing," *arXiv preprint arXiv:2411.04319*, 2024, neurIPS 2024. [Online]. Available: https://arxiv.org/abs/2411.04319

[43] V. Dibia, "LIDA: A Tool for Automatic Generation of Grammar-Agnostic Visualizations and Infographics using Large Language Models," *arXiv preprint arXiv:2303.02927*, 2023, aCL 2023. [Online]. Available: https://arxiv.org/abs/2303.02927

[44] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou, "Self-Consistency Improves Chain of Thought Reasoning in Language Models," *arXiv preprint arXiv:2203.11171*, 2022, iCLR 2023. [Online]. Available: https://arxiv.org/abs/2203.11171

[45] Y. Geifman and R. El-Yaniv, "Selective Classification for Deep Neural Networks," *arXiv preprint arXiv:1705.08500*, 2017, neurIPS 2017. [Online]. Available: https://arxiv.org/abs/1705.08500

N V V S S Jayakanth Kamisetti (2320090050) is a B.Tech student in Computer Science & Information Technology at Koneru Lakshmaiah Education Foundation, Bowrampet, Hyderabad.

Boddupally Narendar (2320090009) is a B.Tech student in Computer Science & Information Technology at Koneru Lakshmaiah Education Foundation, Bowrampet, Hyderabad.

M Praneeth Reddy (2320090018) is a B.Tech student in Computer Science & Information Technology at Koneru Lakshmaiah Education Foundation, Bowrampet, Hyderabad.